

A note on all-pairs shortest paths with isomorphic priors

James Fox*

September 29, 2026

Abstract

In the *all-pairs shortest-paths* (APSP) problem, we are given a directed graph with real edge weights and want to compute the distance between every ordered pair of vertices. We consider APSP in the *isomorphic-priors model*, recently introduced by Dorfman et al. [DDH+26]. In this model, computation proceeds in two phases. In the *preprocessing* phase, an algorithm receives k prior graphs on n vertices and their APSP solutions. In the *query* phase, the algorithm receives a graph H . If H is isomorphic to some prior graph, then the algorithm must return the correct solution. Otherwise, the algorithm may return the correct solution or report that H is not isomorphic to any prior.

Dorfman et al. conjectured that no algorithm in this model solves APSP with $O(n^{3-\varepsilon})$ preprocessing and query time for any constant $\varepsilon > 0$, even for $k = O(1)$ priors [DDH+26, Conjecture 9.3]. We disprove this conjecture, giving a randomised algorithm that solves APSP with high probability using $\tilde{O}(kn^\omega)$ preprocessing time and $\tilde{O}(n^\omega)$ query time.

1 Introduction

The *all-pairs shortest-paths* (APSP) problem asks for the distance between every ordered pair of vertices in a weighted directed graph. For a graph on n vertices, the Floyd–Warshall algorithm solves APSP in $O(n^3)$ time [Flo62; War62], and the best-known randomised and deterministic algorithms improve this to $n^{3-o(1)}$ time [Wil18; CW21].¹ Whether APSP admits a *truly subcubic* algorithm, with running time $O(n^{3-\varepsilon})$ for some constant $\varepsilon > 0$, is a major open problem in fine-grained complexity.

But what if we receive some additional information about the graph beforehand? Can we reuse previous computations on new inputs? Motivated by this question, Dorfman et al. recently introduced the *isomorphic-priors model* [DDH+26]. In this model, computation proceeds in two phases.

- In the *preprocessing* phase, an algorithm receives k graphs $G_1, \dots, G_k$ on n vertices, called *priors*, together with the solution to a problem for each of these graphs. For example, for APSP, the algorithm is given the distances between all pairs of vertices in each prior graph.
- In the *query* phase, the algorithm receives a *query graph* H . If H is isomorphic to a prior graph, then the algorithm must return a correct solution. Otherwise, if H is not isomorphic to any prior graph, then the algorithm may either return a correct solution or return $\perp$.

In this model, there is a randomised algorithm to detect whether a graph has a triangle of negative total edge weight using $\tilde{O}(kn^\omega)$ preprocessing time and $O(n^\omega)$ query time [DDH+26, Theorem 5.1], where $\omega < 2.38$ is the matrix multiplication exponent² and $\tilde{O}(\cdot)$ hides polylogarithmic factors in n and k . Without priors,

*EPFL, Switzerland. Email: james.fox@epfl.ch. Part of this work was carried out while affiliated with the Max Planck Institute for Informatics, Germany.

¹All running times are in the real RAM model. We assume that storing, adding, and comparing exact real numbers take constant time, and standard arithmetic and bit operations on integers of $O(\log(n+k))$ bits also take constant time. The values $+\infty$ and $-\infty$ are represented by separate symbols.

²The matrix multiplication exponent ω is the smallest number such that, for every constant $\varepsilon > 0$, two $n \times n$ matrices can be multiplied using $O(n^{\omega+\varepsilon})$ field operations. We omit ε from running-time bounds, and write $O(n^\omega)$.

negative triangle detection is equivalent to the APSP problem, in the sense that either both problems have truly subcubic algorithms or neither does [VW18]. However, the classical reduction from APSP to negative triangle detection does not directly give an APSP algorithm in the isomorphic-priors model, as the algorithm may return $\perp$ if the query graph is not isomorphic to any prior.

Dorfman et al. conjectured that there is no algorithm in the isomorphic-priors model that solves APSP using $O(n^{3-\epsilon})$ preprocessing and query time, even for $k = O(1)$ priors [DDH+26, Conjecture 9.3]. If this conjecture were true, then it would imply that there is no truly subcubic algorithm for two-dimensional Weisfeiler–Leman colouring [DDH+26, Observation 9.4]. However, we show that the conjecture does not hold.

Theorem 1 (APSP with isomorphic priors). *Fix integers $n \geq 2$ and $k \geq 1$ and a constant $c > 0$. There exist algorithms Preprocess and Query with the following inputs and guarantees. The algorithm Preprocess receives prior graphs $G_1, \dots, G_k$ on n vertices with real edge weights, together with the distances between all ordered pairs of vertices in each prior. The algorithm Query receives a graph H on n vertices with real edge weights and the output of Preprocess.*

- **Running times:** *The algorithm Preprocess takes $\tilde{O}(kn^\omega)$ time, and Query takes $\tilde{O}(n^\omega)$ time.*
- **Correctness:** *For every fixed query graph H , the following holds with probability at least $1 - 1/n^c$: if H is isomorphic to a prior, then Query returns the distances between all ordered pairs of vertices in H . Otherwise, Query either returns these distances or returns $\perp$.*

All randomness is used by Preprocess, and we assume that the query graph H is fixed independently of this randomness. The graphs may contain negative cycles. In particular, for a graph G with vertex set $V(G)$, the distance $d_G(u, v)$ is the infimum of the weights of walks from u to v , where $u, v \in V(G)$. This distance is $+\infty$ if there is no such walk and $-\infty$ if the walk weights are unbounded below. The algorithm in Theorem 1 returns these infinite distances as well as finite distances.

Proof overview. Assume throughout that all graphs are directed and have $n \geq 2$ vertices. Introduce a variable X_a for each distinct edge weight a that appears in the priors. The query algorithm in Theorem 1 uses polynomial identity testing over a random finite field.

For a graph $G = (V, E, w)$, define the *polynomial adjacency matrix* A_G as the $n \times n$ matrix with entries $(A_G)_{uv} = X_{w(u,v)}$ if $(u, v) \in E$, and $(A_G)_{uv} = 0$ otherwise. Each walk of length i from u to v contributes the product of its edge variables to $(A_G^i)_{uv}$, which we prove in Lemma 4. Hence, the *polynomial walk matrix*

$$W_G := \sum_{i=0}^{2n-1} A_G^i$$

encodes, in each entry $(W_G)_{uv}$, the multisets of edge weights of walks from u to v of length at most $2n - 1$. Therefore, the matrix W_G encodes the solution to APSP in the following sense.

Let G and H be graphs on n vertices. We prove in Theorem 7 that, for all $u, v \in V(G)$ and $x, y \in V(H)$,

$$(W_G)_{uv} = (W_H)_{xy} \implies d_G(u, v) = d_H(x, y).$$

Suppose further that $\pi : V(G) \rightarrow V(H)$ is a graph isomorphism.³ Then $(W_G)_{uv} = (W_H)_{\pi(u)\pi(v)}$ for all vertices $u, v \in V(G)$, which we show in Lemma 6. We can therefore recover the distance between two vertices in the query graph from an equal polynomial walk matrix entry in a prior graph, without finding a graph isomorphism.

³A graph isomorphism is a bijection between vertex sets that preserves edges, nonedges, and edge weights.

However, we cannot write down these polynomials explicitly, as each polynomial $(W_G)_{uv}$ may have exponentially many monomials. Instead, we evaluate W_G over a finite field $\mathbb{F}_p$ for a random prime $p \in [n^2, 2n^2]$, by setting each variable X_a to a uniformly random field element. Using repeated squaring, we can evaluate $W_G = \sum_{i=0}^{2n-1} A_G^i$ in $\tilde{O}(n^\omega)$ time for each graph G .

During preprocessing, we repeat this evaluation independently $r = \Theta(1 + \log k / \log n)$ times, using the same primes and variable values for every prior. For each prior graph G and pair $u, v \in V(G)$, we store the list of r evaluated values of $(W_G)_{uv}$ as a dictionary key, with $d_G(u, v)$ as the associated dictionary value. The independent repetitions reduce the probability of the bad event that polynomial walk matrix entries corresponding to different distances give the same lists of evaluated values. For a query graph H , we then evaluate W_H using the same primes and variable values as during preprocessing. For each $x, y \in V(H)$, we look up the list of evaluated values of $(W_H)_{xy}$ in the dictionary, and return the corresponding distances if these keys exist. We prove in Section 3 that, for each fixed query graph, the algorithm satisfies the correctness guarantee in Theorem 1 with probability at least $1 - 1/n^c$.

Organisation. In Section 2, we define the polynomial walk matrix W_G and evaluate it over a random finite field. In Section 3, we give the preprocessing and query algorithms and prove Theorem 1. In Appendix A, we prove that equal polynomial walk matrix entries imply that the corresponding vertex distances are equal.

2 Walk polynomials

Let $G = (V, E, w)$ be a graph with $n = |V| \geq 2$ vertices and edge set $E \subseteq V \times V$. Each edge $e \in E$ has a real weight w_e . A *walk* in G is a sequence $P = (v_0, \dots, v_\ell)$ of vertices such that $(v_{i-1}, v_i) \in E$ for each $i \in [\ell]$. The walk has length $|P| := \ell$, where repeated occurrences of an edge are counted separately. We write $e \in P$ if $e = (v_{i-1}, v_i)$ for some $i \in [\ell]$. In any sum or product over $e \in P$, each occurrence of an edge is included separately, so the total weight of P is $w(P) := \sum_{e \in P} w_e$. The empty walk (v_0) has length and weight zero.

The *distance* from u to v for vertices $u, v \in V$ is

$$d_G(u, v) := \inf \{w(P) : P \text{ is a walk from } u \text{ to } v\}.$$

In particular, $d_G(u, v) = +\infty$ if and only if there is no walk from u to v . Moreover, $d_G(u, v) = -\infty$ if and only if there is a cycle of negative total weight that is reachable from u and from which v is reachable. In all other cases, $d_G(u, v)$ is finite. In the APSP problem, we want to compute $d_G(u, v)$ for every $u, v \in V$. We encode these distances in a matrix whose entries are polynomials.

2.1 Encoding distances by a polynomial matrix

In the remainder of this section, suppose that all graphs have edge weights in a finite set $\mathcal{W} \subseteq \mathbb{R}$. In the APSP algorithm, we will take $\mathcal{W}$ to be the set of all edge weights in the priors.

For each $a \in \mathcal{W}$, introduce a variable X_a and define $\mathbf{X} := \{X_a : a \in \mathcal{W}\}$. Let $\mathbb{N}[\mathbf{X}]$ denote the set of polynomials in these variables with nonnegative integer coefficients. For example, if the edge weights are $\mathcal{W} = \{1, 3/2\}$, then $\mathbf{X} = \{X_1, X_{3/2}\}$ and $2X_1 + 3X_1X_{3/2}^5 \in \mathbb{N}[\mathbf{X}]$. In this polynomial, 2 and 3 are the *coefficients*, while X_1 and $X_1X_{3/2}^5$ are the *monomials*. The *degree* of a monomial $\prod_{a \in \mathcal{W}} X_a^{i_a}$ is $\sum_{a \in \mathcal{W}} i_a$, so these two monomials have degrees 1 and 6, respectively. The *degree* of a nonzero polynomial is the largest degree of its monomials, and the zero polynomial has degree $-\infty$.

We use the following matrices of polynomials to encode walks in a given graph.

Definition 2 (Polynomial adjacency and walk matrices). Let $G = (V, E, w)$ be a graph on n vertices with edge weights in $\mathcal{W}$. The *polynomial adjacency matrix* $A_G \in \mathbb{N}[\mathbf{X}]^{n \times n}$ has rows and columns indexed by V

and entries

$$(A_G)_{uv} := \begin{cases} X_{w(u,v)}, & \text{if } (u, v) \in E, \\ 0, & \text{otherwise.} \end{cases}$$

The *polynomial walk matrix* $W_G \in \mathbb{N}[\mathbf{X}]^{n \times n}$ is defined by

$$W_G := \sum_{i=0}^{2n-1} A_G^i,$$

where $A_G^0 = I_n$ is the $n \times n$ identity matrix.

The polynomial walk matrix encodes all walks of length at most $2n - 1$ in the graph G , as illustrated by the next example.

Example 3. Let $G = (V, E, w)$ have vertices $V = \{1, 2, 3\}$ and edges $E = \{(1, 2), (1, 3), (2, 3), (3, 1)\}$, with weights $w_{(1,2)} = 8$, $w_{(1,3)} = 7$, $w_{(2,3)} = 9$, and $w_{(3,1)} = 7$. The polynomial adjacency matrix of G is

$$A_G = \begin{pmatrix} (A_G)_{11} & (A_G)_{12} & (A_G)_{13} \\ (A_G)_{21} & (A_G)_{22} & (A_G)_{23} \\ (A_G)_{31} & (A_G)_{32} & (A_G)_{33} \end{pmatrix} = \begin{pmatrix} 0 & X_8 & X_7 \\ 0 & 0 & X_9 \\ X_7 & 0 & 0 \end{pmatrix}.$$

Taking powers gives

$$A_G^2 = \begin{pmatrix} X_7^2 & 0 & X_8 X_9 \\ X_7 X_9 & 0 & 0 \\ 0 & X_7 X_8 & X_7^2 \end{pmatrix}, \quad A_G^3 = \begin{pmatrix} X_7 X_8 X_9 & X_7^2 X_8 & X_7^3 \\ 0 & X_7 X_8 X_9 & X_7^2 X_9 \\ X_7^3 & 0 & X_7 X_8 X_9 \end{pmatrix}.$$

Each walk of length i from u to v contributes the product of its edge variables to $(A_G^i)_{uv}$, which we prove in Lemma 4. For example, the only walk of length three from 1 to 3 is $(1, 3, 1, 3)$. Since $w_{(1,3)} = w_{(3,1)} = 7$, the corresponding entry is $(A_G^3)_{13} = X_7^3$. Similarly, the only walks of length four from 1 to 3 are $(1, 3, 1, 2, 3)$ and $(1, 2, 3, 1, 3)$. Each walk uses two edges of weight 7, one of weight 8, and one of weight 9, so it follows that $(A_G^4)_{13} = 2X_7^2 X_8 X_9$. One can also verify that $(A_G^5)_{13} = X_7^5 + X_7 X_8^2 X_9^2$, and thus

$$(W_G)_{13} = \sum_{i=0}^5 (A_G^i)_{13} = 0 + X_7 + X_8 X_9 + X_7^3 + 2X_7^2 X_8 X_9 + (X_7^5 + X_7 X_8^2 X_9^2).$$

The exponents in each monomial specify the multiset of edge weights occurring in the walk, and the coefficient of the monomial is the number of walks with that multiset of edge weights.

Lemma 4 (Powers of the polynomial adjacency matrix encode walks). Let $G = (V, E, w)$ be a graph with edge weights in $\mathcal{W}$. For every integer $i \geq 0$ and vertices $u, v \in V$,

$$(A_G^i)_{uv} = \sum_{P \text{ is a walk of length } i \text{ from } u \text{ to } v \text{ in } G} \left(\prod_{e \in P} X_{w_e} \right),$$

where the empty product is 1.

Proof. We prove the identity by induction on i . First suppose that $i = 0$. If $u = v$ then both sides are 1, and otherwise, both sides are 0.

Now suppose that the identity holds for some $i \geq 0$. Expanding the matrix multiplication gives

$$(A_G^{i+1})_{uv} = \sum_{x \in V} (A_G^i)_{ux} (A_G)_{xv}.$$

By the inductive hypothesis, we obtain the terms on the right by taking each walk of length i from u to x and adding the edge (x, v) , whenever this edge exists. Every walk of length $i + 1$ from u to v has a unique penultimate vertex x , so we count each walk exactly once, and the result follows. $\square$

Corollary 5 (Degree and coefficients of the polynomial walk matrix). *Let $G = (V, E, w)$ be a graph on n vertices with edge weights in $\mathcal{W}$, and let $u, v \in V$. Then the polynomial $(W_G)_{uv} \in \mathbb{N}[\mathbf{X}]$ has degree at most $2n - 1$, and every coefficient is at most n^{2n-1} .*

Proof. By Lemma 4, every monomial in $(A_G^i)_{uv}$ has degree i . Since $(W_G)_{uv} = \sum_{i=0}^{2n-1} (A_G^i)_{uv}$, the degree of $(W_G)_{uv}$ is at most $2n - 1$. Now fix a monomial of degree i with a nonzero coefficient in $(W_G)_{uv}$. By Lemma 4, its coefficient counts the walks from u to v with the multiset of edge weights specified by its exponents. Every such walk has length i . Starting at u , there are at most n choices for each subsequent vertex, so there are at most n^i such walks. Thus the coefficient is at most $n^i \leq n^{2n-1}$. $\square$

In the isomorphic-priors model, the query graph might be *isomorphic* to one of the prior graphs but not *equal* to any of them. The next result shows that the walk matrix entries behave well under isomorphisms.

Lemma 6 (Walk matrix entries are preserved by isomorphisms). *Let G, H be graphs on n vertices with edge weights in $\mathcal{W}$. If $\pi : V(G) \rightarrow V(H)$ is a graph isomorphism, then*

$$(W_G)_{uv} = (W_H)_{\pi(u)\pi(v)} \quad \text{for all } u, v \in V(G).$$

Proof. Since π preserves edges and their weights, $(A_G)_{uv} = (A_H)_{\pi(u)\pi(v)}$ for all $u, v \in V(G)$. Since π is a bijection, it follows by induction on i that $(A_G^i)_{uv} = (A_H^i)_{\pi(u)\pi(v)}$ for every $i \in \mathbb{N}$ and $u, v \in V(G)$. Hence, by the definition of the polynomial walk matrix,

$$(W_G)_{uv} = \sum_{i=0}^{2n-1} (A_G^i)_{uv} = \sum_{i=0}^{2n-1} (A_H^i)_{\pi(u)\pi(v)} = (W_H)_{\pi(u)\pi(v)}. \quad \square$$

It follows that, if some prior graph G is isomorphic to the query graph H , then these graphs have the same polynomial walk matrices after reindexing the entries of W_H according to the graph isomorphism. However, we do not even know how to decide whether H is isomorphic to some prior graph in polynomial time, let alone construct an explicit isomorphism, so we cannot simply reindex the walk matrices.

Instead, our APSP algorithm compares individual entries of the polynomial walk matrices (without trying to compute an isomorphism). This works due to the following key property: whenever two entries of W_G and W_H are equal, then the corresponding graph distances are equal.

Theorem 7 (Walk matrix entries determine distances). *Let G, H be graphs on n vertices with edge weights in a common finite set $\mathcal{W} \subseteq \mathbb{R}$, using the same variable X_a for each $a \in \mathcal{W}$. Let $u, v \in V(G)$ and $x, y \in V(H)$. If $(W_G)_{uv} = (W_H)_{xy}$, then $d_G(u, v) = d_H(x, y)$.*

Proof. The result follows by applying the Cayley–Hamilton theorem to A_G and A_H and then considering the cases for finite and infinite distances separately; a full proof is given in Appendix A. $\square$

2.2 Evaluating walk matrices over a finite field

By Theorem 7, if two graphs have identical polynomial walk matrices (up to reindexing), then they have identical APSP solutions. But there is a bigger problem: each entry of W_G can contain exponentially many monomials, as there may be exponentially many walks of length at most $2n - 1$ between two given vertices. Therefore we cannot write these entries as explicit sums of monomials in polynomial time. Instead, we assign a random finite field element to each variable X_a and compute the resulting numerical matrix.

Let n be sufficiently large, and let $\mathcal{P}$ be the set of primes in $[n^2, 2n^2]$. For a finite nonempty set S , write $z \sim \text{Unif}(S)$ to mean that z is sampled uniformly from S . Sample a random prime $p \sim \text{Unif}(\mathcal{P})$, and let $\mathbb{F}_p$ be the field of integers modulo p , represented by $\{0, 1, \dots, p-1\}$.

Conditional on p , independently sample $x_a \sim \text{Unif}(\mathbb{F}_p)$ for each edge weight $a \in \mathcal{W}$, and let $\mathbf{x} := (x_a)_{a \in \mathcal{W}}$. For a polynomial Q with integer coefficients and variables in $\mathbf{X}$, the *evaluated polynomial* $Q(\mathbf{x}) \in \mathbb{F}_p$ is obtained by replacing each variable X_a by x_a and performing arithmetic modulo p . For a polynomial matrix, evaluation is entrywise; in particular, we write $W_G(\mathbf{x}) \in \mathbb{F}_p^{n \times n}$ for the evaluated walk matrix.

Lemma 8 (Evaluating a walk matrix). *Let G be a graph on n vertices, and $\mathcal{W} \subseteq \mathbb{R}$ be the set of distinct edge weights in G . Let $p \in \mathcal{P}$ be a prime. Given G , p , and an assignment $\mathbf{x} = (x_a)_{a \in \mathcal{W}} \in \mathbb{F}_p^{\mathcal{W}}$, we can compute $W_G(\mathbf{x})$ in $\tilde{O}(n^\omega)$ time.*

Proof. Recall that $W_G = \sum_{i=0}^{2n-1} A_G^i$. Since evaluation commutes with addition and multiplication,

$$W_G(\mathbf{x}) = \sum_{i=0}^{2n-1} A_G^i(\mathbf{x}) = \sum_{i=0}^{2n-1} (A_G(\mathbf{x}))^i.$$

To construct the evaluated adjacency matrix $A_G(\mathbf{x}) \in \mathbb{F}_p^{n \times n}$, first sort the pairs (a, x_a) for each $a \in \mathcal{W}$ by their first coordinate. This takes $O(n^2 \log n)$ time. For each edge (u, v) , we use binary search to find its assigned value $(A_G(\mathbf{x}))_{uv} = x_{w(u,v)}$ in $O(\log n)$ time. All remaining entries of $A_G(\mathbf{x})$ are 0, so we can construct $A_G(\mathbf{x})$ using $O(n^2 \log n)$ time.

Next, we compute $W_G(\mathbf{x}) = \sum_{i=0}^{2n-1} (A_G(\mathbf{x}))^i$ by repeatedly squaring $A_G(\mathbf{x})$ and computing the partial sums at the same time. This uses $O(\log n)$ matrix additions and multiplications over $\mathbb{F}_p$. Since $p \leq 2n^2$ and $\omega \geq 2$, including the construction of $A_G(\mathbf{x})$, the total time is therefore $\tilde{O}(n^\omega)$. $\square$

Let G, H be distinct graphs and let $u, v \in V(G)$ and $x, y \in V(H)$. By Theorem 7, if the unevaluated polynomial walk matrix entries satisfy $(W_G)_{uv} = (W_H)_{xy}$, then $d_G(u, v) = d_H(x, y)$. After evaluation, if $(W_G(\mathbf{x}))_{uv} = (W_H(\mathbf{x}))_{xy}$, then we might hope that $d_G(u, v) = d_H(x, y)$ so that we can use the evaluated walk matrices to certify the APSP solutions. However, this need not hold.

The next result bounds the probability of the “bad” event that $(W_G(\mathbf{x}))_{uv} = (W_H(\mathbf{x}))_{xy}$ when two pairs of vertices have distinct distances; we call such an event a *bad collision*.

Lemma 9 (Probability of a bad collision). *Let G, H be graphs on n vertices with edge weights in $\mathcal{W} \subseteq \mathbb{R}$. Let $u, v \in V(G)$ and $x, y \in V(H)$. Sample $p \sim \text{Unif}(\mathcal{P})$, where $\mathcal{P}$ is the set of primes in $[n^2, 2n^2]$. Conditional on p , independently sample $x_a \sim \text{Unif}(\mathbb{F}_p)$ for each $a \in \mathcal{W}$, and let $\mathbf{x} := (x_a)_{a \in \mathcal{W}}$. There is an absolute constant $C > 0$ such that, if $d_G(u, v) \neq d_H(x, y)$, then*

$$\Pr\left[(W_G(\mathbf{x}))_{uv} = (W_H(\mathbf{x}))_{xy}\right] \leq \frac{C \log n}{n},$$

where the probability is over the choice of p and $\mathbf{x}$.

Proof. Suppose that $d_G(u, v) \neq d_H(x, y)$. Then by the contrapositive of Theorem 7, $(W_G)_{uv} \neq (W_H)_{xy}$. Thus $Q := (W_G)_{uv} - (W_H)_{xy}$ is a nonzero polynomial with integer coefficients and

$$\Pr\left[(W_G(\mathbf{x}))_{uv} = (W_H(\mathbf{x}))_{xy}\right] = \Pr[Q(\mathbf{x}) = 0].$$

Fix a nonzero coefficient α of Q , and let $\mathcal{E}_p$ be the event that $\alpha \equiv 0 \pmod{p}$ over the random choice of p . Then conditioning on $\mathcal{E}_p$ gives

$$\begin{aligned} \Pr[Q(\mathbf{x}) = 0] &= \Pr[Q(\mathbf{x}) = 0 \text{ and } \mathcal{E}_p] + \Pr[Q(\mathbf{x}) = 0 \mid \neg \mathcal{E}_p] \Pr[\neg \mathcal{E}_p] \\ &\leq \Pr[\mathcal{E}_p] + \Pr[Q(\mathbf{x}) = 0 \mid \neg \mathcal{E}_p]. \end{aligned}$$

We first bound $\Pr[\mathcal{E}_p]$. By Corollary 5, $|\alpha| \leq n^{2n-1}$. Suppose for contradiction that $\alpha \equiv 0 \pmod{p_j}$ for n distinct primes $p_1, \dots, p_n \in \mathcal{P}$. Then $\prod_{j=1}^n p_j$ divides α . Since each prime in $\mathcal{P}$ is at least n^2 , it follows that $n^{2n} \leq \prod_{j=1}^n p_j \leq |\alpha| \leq n^{2n-1}$, a contradiction. Hence fewer than n primes in $\mathcal{P}$ satisfy $\alpha \equiv 0 \pmod{p}$. Since $p \sim \text{Unif}(\mathcal{P})$ and $|\mathcal{P}| = \Theta(n^2/\log n)$ by the prime number theorem, it follows that

$$\Pr[\mathcal{E}_p] = \Pr[\alpha \equiv 0 \pmod{p}] \leq \frac{n}{|\mathcal{P}|} = O\left(\frac{\log n}{n}\right).$$

Next, fix a prime $p \in \mathcal{P}$ such that $\alpha \not\equiv 0 \pmod{p}$. Then Q is nonzero over $\mathbb{F}_p$ and has degree at most $2n - 1$ by Corollary 5. By the Schwartz–Zippel lemma, the probability over $\mathbf{x}$ that $Q(\mathbf{x}) = 0$ for this prime p is at most $(2n - 1)/p < 2n/n^2 = 2/n$. This bound holds for every prime satisfying $\neg\mathcal{E}_p$, so it follows that

$$\Pr[Q(\mathbf{x}) = 0 \mid \neg\mathcal{E}_p] < \frac{2}{n} = O\left(\frac{1}{n}\right),$$

where the probability is over p and $\mathbf{x}$. Combining the two bounds gives

$$\Pr[Q(\mathbf{x}) = 0] \leq O\left(\frac{\log n}{n}\right) + O\left(\frac{1}{n}\right) = O\left(\frac{\log n}{n}\right).$$

Hence, there is an absolute constant $C > 0$ such that $\Pr[Q(\mathbf{x}) = 0] \leq C \log n/n$. $\square$

3 The APSP algorithm

We now use the polynomials from the previous section to solve APSP in the isomorphic-priors model with probability at least $1 - 1/n^c$ for a given constant $c > 0$. Assume in this section that n is sufficiently large; for bounded values of n , we can solve APSP directly. The preprocessing algorithm is given in Algorithm 1, and the query algorithm is given in Algorithm 2.

3.1 The preprocessing algorithm

To reduce the probability of bad collisions, we carry out r independent repetitions of the random evaluation from Section 2.2. Let $C > 0$ be the constant from Lemma 9. Choose a positive integer r such that

$$r = \Theta\left(1 + \frac{\log k}{\log n}\right) \quad \text{and} \quad \left(\frac{C \log n}{n}\right)^r \leq \frac{1}{k^2 n^{c+4}}, \quad (3.1)$$

where the constant hidden by Θ is sufficiently large and depends only on c and C . Such a choice is possible because $\log(n/(C \log n)) = \Theta(\log n)$. We choose this value of r as there are $\binom{kn^2}{2}$ unordered pairs of prior matrix entries and kn^4 pairs consisting of a prior matrix entry and an entry of a fixed query matrix. This choice allows for a union bound for both counts in the proofs of Lemmas 10 and 11.

In the preprocessing algorithm, we repeat the sampling procedure from Section 2.2 independently r times, obtaining primes $p_1, \dots, p_r$ and values $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(r)}$. For each prior graph G_i and each pair of vertices $u, v \in V(G_i)$, we compute the polynomial walk matrix entry $(W_{G_i})_{uv}$ under each evaluation and store the resulting vector

$$h_{G_i}(u, v) := \left((W_{G_i}(\mathbf{x}^{(1)}))_{uv}, \dots, (W_{G_i}(\mathbf{x}^{(r)}))_{uv} \right) \in \mathbb{F}_{p_1} \times \dots \times \mathbb{F}_{p_r}.$$

We then create a dictionary D in which the keys are the vectors $h_{G_i}(u, v)$ and the stored values are the corresponding distances $d_{G_i}(u, v)$. If two equal keys have different distances, then the preprocessing algorithm returns $\perp$. The full algorithm is given in Algorithm 1 and analysed in Lemma 10.

Algorithm 1 Preprocess

Input: Prior graphs $G_1, \dots, G_k$ on n vertices and distances $d_{G_i}(u, v)$ for every $i \in [k]$ and $u, v \in V(G_i)$.

Output: Either $\perp$, or the weights $\mathcal{W}$, dictionary D , and sampled primes p_j and values $\mathbf{x}^{(j)}$ for $j \in [r]$.

- 1: Form the set $\mathcal{W}$ of distinct edge weights of $G_1, \dots, G_k$.
- 2: Compute the set $\mathcal{P}$ of primes in $[n^2, 2n^2]$.
- 3: **for** $j = 1, \dots, r$ **do**
- 4: Sample $p_j \sim \text{Unif}(\mathcal{P})$.
- 5: Independently sample $x_a^{(j)} \sim \text{Unif}(\mathbb{F}_{p_j})$ for each $a \in \mathcal{W}$, and let $\mathbf{x}^{(j)} := (x_a^{(j)})_{a \in \mathcal{W}}$.
- 6: For every $i \in [k]$ and $u, v \in V(G_i)$, compute

$$h_{G_i}(u, v) := \left((W_{G_i}(\mathbf{x}^{(1)}))_{uv}, \dots, (W_{G_i}(\mathbf{x}^{(r)}))_{uv} \right).$$

- 7: Initialise an empty dictionary D .
 - 8: **for** each $i \in [k]$ and $u, v \in V(G_i)$ **do**
 - 9: **if** D contains $h_{G_i}(u, v)$ and $D[h_{G_i}(u, v)] \neq d_{G_i}(u, v)$ **then**
 - 10: **return** $\perp$.
 - 11: Set $D[h_{G_i}(u, v)] \leftarrow d_{G_i}(u, v)$.
 - 12: **return** $\mathcal{W}$, D , and $p_j, \mathbf{x}^{(j)}$ for $j \in [r]$.
-

Lemma 10 (Preprocessing time and failure probability). *The preprocessing algorithm in Algorithm 1 runs in $\tilde{O}(kn^\omega)$ time and returns $\perp$ with probability at most $1/n^c$.*

Proof. Fix $a, b \in [k]$ and vertices $u, v \in V(G_a)$ and $x, y \in V(G_b)$. Suppose that $d_{G_a}(u, v) \neq d_{G_b}(x, y)$. By Lemma 9, for each $j \in [r]$, the probability of a bad collision is

$$\Pr \left[(W_{G_a}(\mathbf{x}^{(j)}))_{uv} = (W_{G_b}(\mathbf{x}^{(j)}))_{xy} \right] \leq \frac{C \log n}{n}.$$

Since the r repetitions are independent, it follows by (3.1) that

$$\Pr[h_{G_a}(u, v) = h_{G_b}(x, y)] \leq \left(\frac{C \log n}{n} \right)^r \leq \frac{1}{k^2 n^{c+4}}.$$

The preprocessing algorithm returns $\perp$ only if there are two priors G_a, G_b such that, for some $u, v \in V(G_a)$ and $x, y \in V(G_b)$, we have $h_{G_a}(u, v) = h_{G_b}(x, y)$ but $d_{G_a}(u, v) \neq d_{G_b}(x, y)$. There are k priors, each with n^2 ordered pairs of vertices, giving kn^2 vectors $h_{G_a}(u, v)$. A union bound over their $\binom{kn^2}{2}$ unordered pairs of such vectors gives

$$\Pr[\text{Preprocess returns } \perp] \leq \binom{kn^2}{2} \frac{1}{k^2 n^{c+4}} \leq \frac{1}{n^c}.$$

For the running time, first sort the distinct edge weights and store the position of each edge weight in the sorted list. This takes $O(kn^2 \log(kn))$ time. The set $\mathcal{P}$ can be computed in $\tilde{O}(n^2)$ time using the sieve of Eratosthenes. Sampling and storing p_j and $\mathbf{x}^{(j)}$ for all $j \in [r]$ takes $O(r(1 + |\mathcal{W}|)) = O(rkn^2)$ time. For each $i \in [k]$ and $j \in [r]$, computing $W_{G_i}(\mathbf{x}^{(j)})$ takes $\tilde{O}(n^\omega)$ time by Lemma 8. Thus all rk evaluated matrices, and hence the vectors $h_{G_i}(u, v)$, can be computed in $\tilde{O}(rkn^\omega)$ time.

We implement the dictionary D as a balanced search tree, ordering the vectors $h_{G_i}(u, v)$ lexicographically. Each vector has r values, so comparing two vectors takes $O(r)$ time. Inserting all kn^2 vectors therefore takes $O(rkn^2 \log(kn))$ time. Since $\omega \geq 2$ and $r = \Theta(1 + \log k / \log n)$, the total time is $\tilde{O}(kn^\omega)$. $\square$

3.2 The query algorithm

Fix a query graph H independently of the randomness used during preprocessing. If Preprocess returns $\perp$, then the query algorithm also returns $\perp$. Suppose that Preprocess succeeds and returns the set $\mathcal{W}$ of prior edge weights, a dictionary D , and sampled primes p_j and values $\mathbf{x}^{(j)}$ for each $j \in [r]$.

If H contains an edge weight that is not in $\mathcal{W}$, then H is not isomorphic to any prior, so the query algorithm returns $\perp$. Otherwise, we compute the vectors $h_H(x, y)$ for each $x, y \in V(H)$ using the same primes p_j and values $\mathbf{x}^{(j)}$ as during preprocessing. We then look up each key $h_H(x, y)$ in the dictionary D . If all keys are present in D , then the query algorithm returns the distance estimates $\hat{d}_H(x, y) := D[h_H(x, y)]$ for all $x, y \in V(H)$. Otherwise, the query algorithm returns $\perp$. The full algorithm is given in Algorithm 2 and analysed in Lemma 11.

Algorithm 2 Query

Input: A graph H on n vertices and the output of Preprocess.

Output: Either $\perp$, or a distance estimate $\hat{d}_H(x, y)$ for every $x, y \in V(H)$.

- 1: **if** Preprocess returned $\perp$ **then**
 - 2: **return** $\perp$.
 - 3: **if** some edge weight of H is not in $\mathcal{W}$ **then** $\triangleright H$ is not isomorphic to any prior.
 - 4: **return** $\perp$.
 - 5: For every $x, y \in V(H)$, use the primes p_j and values $\mathbf{x}^{(j)}$ from Preprocess to compute
$$h_H(x, y) := \left((W_H(\mathbf{x}^{(1)}))_{xy}, \dots, (W_H(\mathbf{x}^{(r)}))_{xy} \right).$$
 - 6: **if** D does not contain the key $h_H(x, y)$ for some $x, y \in V(H)$ **then**
 - 7: **return** $\perp$.
 - 8: **return** a distance estimate $\hat{d}_H(x, y) := D[h_H(x, y)]$ for every $x, y \in V(H)$.
-

Lemma 11 (Query time and error probability). *Let H be a query graph on n vertices, fixed independently of the randomness of Preprocess. Given the output of Preprocess, the algorithm Query takes $\tilde{O}(n^\omega)$ time and has the following properties.*

- **Correctness:** *if H is isomorphic to some prior graph and Preprocess does not return $\perp$, then Query returns distance estimates satisfying*

$$\hat{d}_H(x, y) = d_H(x, y) \quad \text{for all } x, y \in V(H).$$

- **Error probability:** *the probability over the randomness of Preprocess that Query returns an incorrect distance $\hat{d}_H(x, y) \neq d_H(x, y)$ for some $x, y \in V(H)$ is at most $1/n^c$.*

Proof. For the running time, checking whether the output of Preprocess is $\perp$ takes $O(1)$ time. If not, then we can check that each edge weight of H appears in $\mathcal{W}$ using binary search. Retrieving the r assigned values for every distinct query edge weight then takes $O(rn^2)$ time. By Lemma 8, these assignments suffice to compute all r evaluated query matrices for H in $\tilde{O}(rn^\omega)$ time. It remains to prove the two correctness guarantees.

Suppose that Preprocess does not return $\perp$ and that H is isomorphic to some prior G_i under the graph isomorphism $\pi : V(G_i) \rightarrow V(H)$. Every edge weight of G_i is in $\mathcal{W}$, so every edge weight of H is in $\mathcal{W}$. By Lemma 6, $h_H(\pi(u), \pi(v)) = h_{G_i}(u, v)$ for all $u, v \in V(G_i)$. Since π is a bijection, the dictionary D contains $h_H(x, y)$ for every $x, y \in V(H)$. The key $h_{G_i}(u, v)$ was inserted into D with value $d_{G_i}(u, v)$ during preprocessing. Hence, for all $u, v \in V(G_i)$,

$$\hat{d}_H(\pi(u), \pi(v)) = D[h_{G_i}(u, v)] = d_{G_i}(u, v) = d_H(\pi(u), \pi(v)).$$

Therefore, as π is a bijection, Query returns the correct distance for every ordered pair of vertices of H .

For the error probability, fix H independently of the preprocessing randomness. If H has an edge weight outside $\mathcal{W}$, then Query returns $\perp$, so assume that all edge weights of H belong to $\mathcal{W}$. An incorrect distance can be returned only when Preprocess does not return $\perp$, but we bound its probability over the original preprocessing randomness, without conditioning on the output of Preprocess.

Fix $i \in [k]$ and vertices $u, v \in V(G_i)$ and $x, y \in V(H)$ such that $d_{G_i}(u, v) \neq d_H(x, y)$. By Lemma 9, independence of the repetitions, and the choice of r in (3.1),

$$\Pr[h_{G_i}(u, v) = h_H(x, y)] \leq \left(\frac{C \log n}{n} \right)^r \leq \frac{1}{k^2 n^{c+4}}.$$

If Query returns an incorrect estimate $\hat{d}_H(x, y)$, then some prior pair $u, v \in V(G_i)$ satisfies

$$h_{G_i}(u, v) = h_H(x, y) \quad \text{and} \quad d_{G_i}(u, v) = \hat{d}_H(x, y) \neq d_H(x, y).$$

There are kn^2 prior pairs and n^2 query pairs, so a union bound gives

$$\Pr[\text{Query returns an incorrect distance estimate}] \leq kn^4 \frac{1}{k^2 n^{c+4}} \leq \frac{1}{n^c}.$$

□

We now use the two failure probability bounds in Lemmas 10 and 11 to prove Theorem 1.

Theorem 1 (APSP with isomorphic priors). *Fix integers $n \geq 2$ and $k \geq 1$ and a constant $c > 0$. There exist algorithms Preprocess and Query with the following inputs and guarantees. The algorithm Preprocess receives prior graphs $G_1, \dots, G_k$ on n vertices with real edge weights, together with the distances between all ordered pairs of vertices in each prior. The algorithm Query receives a graph H on n vertices with real edge weights and the output of Preprocess.*

- **Running times:** *The algorithm Preprocess takes $\tilde{O}(kn^\omega)$ time, and Query takes $\tilde{O}(n^\omega)$ time.*
- **Correctness:** *For every fixed query graph H , the following holds with probability at least $1 - 1/n^c$: if H is isomorphic to a prior, then Query returns the distances between all ordered pairs of vertices in H . Otherwise, Query either returns these distances or returns $\perp$.*

Proof. For small n , we can solve APSP directly using the Floyd–Warshall algorithm. Suppose that n is sufficiently large. By Lemmas 10 and 11, the algorithm Preprocess runs in $\tilde{O}(kn^\omega)$ time, and the algorithm Query runs in $\tilde{O}(n^\omega)$ time. It remains to prove the correctness guarantee. By Lemma 10, Preprocess returns $\perp$ with probability at most $1/n^c$. Fix a query graph H independently of the preprocessing randomness.

If H is isomorphic to some prior graph, then by Lemma 11, the output of Query is correct whenever Preprocess does not return $\perp$, which occurs with probability at least $1 - 1/n^c$. Suppose that H is not isomorphic to any prior. If Preprocess returns $\perp$, then Query returns $\perp$, which is a valid answer. If Query does not return $\perp$, then the output of Query is incorrect only if some estimated distance is incorrect. By Lemma 11, this occurs with probability at most $1/n^c$.

Thus, in all cases, Query returns a valid answer with probability at least $1 - 1/n^c$. □

AI disclosure. ChatGPT contributed materially to the proofs and exposition in this work, as well as finding references. Initially, GPT-5.6 Sol suggested using the polynomial adjacency matrix A_G as part of a complicated APSP certificate involving four different polynomials. I used this to develop a simpler construction using two polynomials: $\sum_{i=0}^{n-1} A_G^i$ to encode finite distances and $\sum_{i=1}^n A_G^i$ (with a separate set of variables) to detect negative cycles. Martin Herold then suggested that a single polynomial would suffice and, starting from his observations, I developed the proof above in conversation with GPT-6 Astra. I checked every proof and reference and take responsibility for the contents.

Acknowledgements. I would like to thank Martin Herold for taking the time to explain the isomorphic-priors model and suggesting several ways of simplifying the proof.

References

- [CW21] Timothy M. Chan and R. Ryan Williams. “Deterministic APSP, Orthogonal Vectors, and More: Quickly Derandomizing Razborov–Smolensky”. In: *ACM Transactions on Algorithms* 17.1 (2021), 2:1–2:14. DOI: [10.1145/3402926](https://doi.org/10.1145/3402926).
- [DDH+26] Dani Dorfman, Simon Döring, Martin Herold, Danupon Nanongkai, Daniel Neuen, Joachim Spoerhase, and Zihang Wu. “Can We Break Fine-Grained and NP-Hardness Barriers if We’ve Seen the Graph Before? The Isomorphic-Priors Model”. In: *Proceedings of the 67th IEEE Symposium on Foundations of Computer Science (FOCS 2026)*. To appear. 2026.
- [Flo62] Robert W. Floyd. “Algorithm 97: Shortest Path”. In: *Communications of the ACM* 5.6 (1962), p. 345. DOI: [10.1145/367766.368168](https://doi.org/10.1145/367766.368168).
- [VW18] Virginia Vassilevska Williams and R. Ryan Williams. “Subcubic Equivalences Between Path, Matrix, and Triangle Problems”. In: *Journal of the ACM* 65.5 (2018), 27:1–27:38. DOI: [10.1145/3186893](https://doi.org/10.1145/3186893).
- [War62] Stephen Warshall. “A Theorem on Boolean Matrices”. In: *Journal of the ACM* 9.1 (1962), pp. 11–12. DOI: [10.1145/321105.321107](https://doi.org/10.1145/321105.321107).
- [Wil18] R. Ryan Williams. “Faster All-Pairs Shortest Paths via Circuit Complexity”. In: *SIAM Journal on Computing* 47.5 (2018), pp. 1965–1985. DOI: [10.1137/15M1024524](https://doi.org/10.1137/15M1024524).

A Walk matrices encode distances

This appendix contains the proof of Theorem 7. We first show in Lemma 12 that, if two polynomial walk matrix entries are equal, then the corresponding entries of every power of the polynomial adjacency matrices are equal. This allows us to extend the equality in Theorem 7 from walks of length at most $2n - 1$ to walks of any length, including negative cycles.

Lemma 12 (Equal walk matrix entries imply equal entries of all adjacency matrix powers). *Let G, H be graphs on n vertices with edge weights in a finite set $\mathcal{W} \subseteq \mathbb{R}$. Let $u, v \in V(G)$ and $x, y \in V(H)$. If $(W_G)_{uv} = (W_H)_{xy}$, then*

$$(A_G^i)_{uv} = (A_H^i)_{xy} \quad \text{for every } i \in \mathbb{N}.$$

Proof. We prove the result by induction on i , using the Cayley–Hamilton theorem. Denote the *characteristic polynomials* of A_G and A_H by $\chi_G(t) := \det(tI_n - A_G)$ and $\chi_H(t) := \det(tI_n - A_H)$, where t is a variable and I_n is the $n \times n$ identity matrix. Define $q(t) := \chi_G(t) \chi_H(t)$. Both $\chi_G(t)$ and $\chi_H(t)$ have degree n in t and leading coefficient 1, so we can write

$$q(t) = t^{2n} - \sum_{j=0}^{2n-1} c_j t^j,$$

where $c_0, \dots, c_{2n-1} \in \mathbb{Z}[\mathbf{X}]$. By the Cayley–Hamilton theorem, $\chi_G(A_G) = \chi_H(A_H) = 0$, and it follows that $q(A_G) = q(A_H) = 0$. Thus, for all $i \geq 2n$,

$$A_G^i = A_G^{i-2n} A_G^{2n} = A_G^{i-2n} \left(q(A_G) + \sum_{j=0}^{2n-1} c_j A_G^j \right) = \sum_{j=0}^{2n-1} c_j A_G^{i+j-2n}. \quad (\text{A.1})$$

The same calculation for H gives

$$A_H^i = \sum_{j=0}^{2n-1} c_j A_H^{i+j-2n}. \quad (\text{A.2})$$

Now suppose that $(W_G)_{uv} = (W_H)_{xy}$. By Lemma 4, every monomial in each entry of A_G^i or A_H^i has total degree i . Since $(W_G)_{uv} = (W_H)_{xy}$, comparing terms of each degree gives $(A_G^i)_{uv} = (A_H^i)_{xy}$ for $0 \leq i < 2n$.

We extend this equality to all $i \in \mathbb{N}$ by induction on i . Fix $i \geq 2n$ and suppose that $(A_G^j)_{uv} = (A_H^j)_{xy}$ for every $0 \leq j < i$. Then by (A.1), the inductive hypothesis, and (A.2),

$$(A_G^i)_{uv} = \sum_{j=0}^{2n-1} c_j (A_G^{i+j-2n})_{uv} = \sum_{j=0}^{2n-1} c_j (A_H^{i+j-2n})_{xy} = (A_H^i)_{xy}.$$

Thus, $(A_G^i)_{uv} = (A_H^i)_{xy}$ for all $i \in \mathbb{N}$. □

Recall that for two vertices $u, v \in V(G)$, their distance satisfies $d_G(u, v) = -\infty$ if and only if there is a cycle of negative total weight that is reachable from u and from which v is reachable. We now use Lemma 12 to handle the case with negative cycles in the proof of Theorem 7.

Theorem 7 (Walk matrix entries determine distances). *Let G, H be graphs on n vertices with edge weights in a common finite set $\mathcal{W} \subseteq \mathbb{R}$, using the same variable X_a for each $a \in \mathcal{W}$. Let $u, v \in V(G)$ and $x, y \in V(H)$. If $(W_G)_{uv} = (W_H)_{xy}$, then $d_G(u, v) = d_H(x, y)$.*

Proof. Suppose that $(W_G)_{uv} = (W_H)_{xy}$. We consider infinite and finite distances separately.

First, suppose that $d_G(u, v) = +\infty$. Then there is no walk from u to v in G , so $(W_G)_{uv} = 0$. Suppose for contradiction that there is a walk from x to y in H . Then H contains a simple path P from x to y with $|P| \leq n - 1$. By Lemma 4, the monomial $\prod_{e \in P} X_{w_e}$ has a positive coefficient in $(A_H^{|P|})_{xy}$. All coefficients in the sum defining W_H are nonnegative, so this monomial also has a positive coefficient in $(W_H)_{xy}$. This contradicts $(W_H)_{xy} = (W_G)_{uv} = 0$. Hence $d_H(x, y) = +\infty = d_G(u, v)$.

Next, suppose that $d_G(u, v) = -\infty$, and let P be any walk from u to v in G . By Lemma 4, the monomial $\prod_{e \in P} X_{w_e}$ has a positive coefficient in $(A_G^{|P|})_{uv}$. By Lemma 12, this monomial has the same coefficient in $(A_H^{|P|})_{xy}$. By Lemma 4, there is a walk from x to y in H with the same number of occurrences of each edge weight as P , and hence with the same total weight. Since the weights of walks from u to v in G are unbounded below, the weights of walks from x to y in H are also unbounded below. Thus $d_H(x, y) = -\infty = d_G(u, v)$.

Finally, suppose that $d_G(u, v)$ is finite. The preceding arguments also apply with the two vertex pairs interchanged, so it follows that $d_H(x, y) \notin \{\pm\infty\}$. Since $d_G(u, v)$ is finite, every closed subwalk of a walk from u to v has nonnegative weight, so deleting closed subwalks never increases the total weight. Hence there exists a simple path P from u to v in G with $w(P) = d_G(u, v)$ and $|P| \leq n - 1$. By Lemma 4, the monomial $\prod_{e \in P} X_{w_e}$ has a positive coefficient in $(W_G)_{uv}$ and hence in $(W_H)_{xy}$. Thus by Lemma 4, there is a walk from x to y in H with the same number of occurrences of each edge weight as P . The weight of this walk is $d_G(u, v)$, so we have $d_H(x, y) \leq d_G(u, v)$. Swapping the two vertex pairs gives $d_G(u, v) \leq d_H(x, y)$, and therefore $d_G(u, v) = d_H(x, y)$. □